

Chapter 5: Lists

Study Notes

At the end of Chapter 4 you met `split()`, a string method that breaks a sentence into pieces and hands them back as a **list**. That was your first real encounter with this chapter's data type. A list is Python's general-purpose container for storing many values together, in order, under a single variable name. Lists share a lot of behavior with strings -- indexing, slicing, `len()`, and the `in` / `not in` operators all work the same way -- but unlike strings, lists can be changed after they are created.

What you will learn in this chapter:

- Creating lists (with mixed data types) and indexing into them
- List mutability -- how lists differ from the immutable strings of Chapter 4
- List slicing, reusing the `start:stop:step` syntax from Chapter 4
- Core list methods: `append`, `insert`, `remove`, `pop`, `sort`, `reverse`, and more
- The `in` / `not in` membership operators on lists
- Nested lists and 2D-style indexing
- Iterating over a list with a `for` loop, plus a first look at `enumerate()`
- List comprehension -- a compact shorthand for building lists

1. Creating Lists and Accessing Elements

A **list** is written as a comma-separated sequence of values inside square brackets `[]`. A single list can mix different data types -- numbers, strings, booleans, even other lists -- in any combination.

Example

```
fruits = ["apple", "banana", "cherry", "date"]
mixed = ["Maya", 16, True, 5.5]

print(fruits)
print(mixed)
print(type(fruits))
```

Output

```
['apple', 'banana', 'cherry', 'date']
['Maya', 16, True, 5.5]
<class 'list'>
```

Just like a string, a list is an ordered sequence, so every element has an index. Indexing a list uses exactly the same square-bracket syntax you used for strings in Chapter 4.

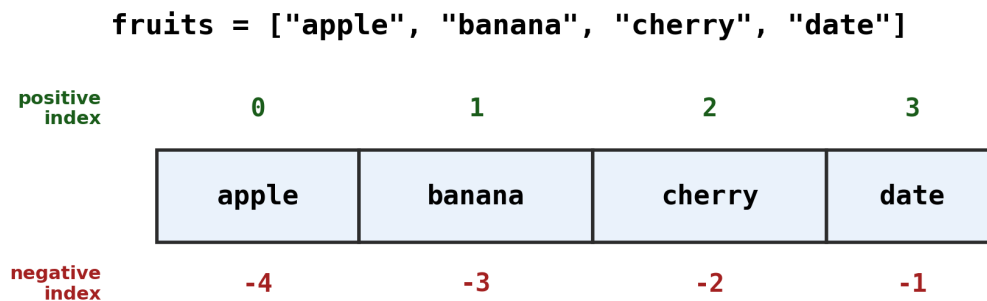


Figure 1: Positive indices count from the left starting at 0; negative indices count from the right starting at -1 -- the same rule that applied to string indexing in Chapter 4.

Example

```
fruits = ["apple", "banana", "cherry", "date"]

print(fruits[0])    # first element
print(fruits[2])    # third element
print(fruits[-1])   # last element
print(len(fruits))  # number of elements
```

Output

```
apple
cherry
date
4
```

The `len()` function works on lists exactly as it did on strings in Chapter 4 -- it simply counts how many items are in the sequence, except now it counts elements instead of characters.

2. List Mutability

Recall from Chapter 4 that strings are **immutable** -- once created, their characters cannot be changed, and `s[0] = "J"` raises a `TypeError`. Lists are the opposite: they are **mutable**, meaning you CAN change, add, or remove elements after the list has been created, without building a brand new list.

Modifying an element by index

```
fruits = ["apple", "banana", "cherry", "date"]
fruits[1] = "blueberry"
print(fruits)
```

Output

```
['apple', 'blueberry', 'cherry', 'date']
```

Strings vs. Lists

This is the single biggest difference between the two sequence types you have met so far. `s[0] = "J"` crashes for a string but `fruits[1] = "blueberry"` works perfectly for a list, because lists were designed to be modified in place after creation.

3. List Slicing

List slicing reuses the exact same `list[start:stop:step]` syntax you learned for strings in Chapter 4: **start** is included, **stop** is excluded, and **step** controls how many elements to skip.

Example

```
nums = [10, 20, 30, 40, 50, 60]

print(nums[1:4])      # elements at index 1, 2, 3
print(nums[:3])       # from the start up to (not including) index 3
print(nums[3:])       # from index 3 to the end
print(nums[:])        # the whole list (a shallow copy)
print(nums[::-1])     # the whole list, reversed
print(nums[::2])      # every 2nd element
```

Output

```
[20, 30, 40]
[10, 20, 30]
[40, 50, 60]
[10, 20, 30, 40, 50, 60]
[60, 50, 40, 30, 20, 10]
[10, 30, 50]
```

Tip

Just like with strings, `nums[:]` does not return the SAME list object -- it builds a new (shallow) copy. This becomes important in Section 8 (Common Beginner Mistakes), where copying lists incorrectly is a frequent source of bugs.

4. List Methods

Because lists are mutable, they come with many methods that change the list directly ("in place") rather than returning a new one. This is a key difference from the string methods in Chapter 4, which

always returned a brand new string.

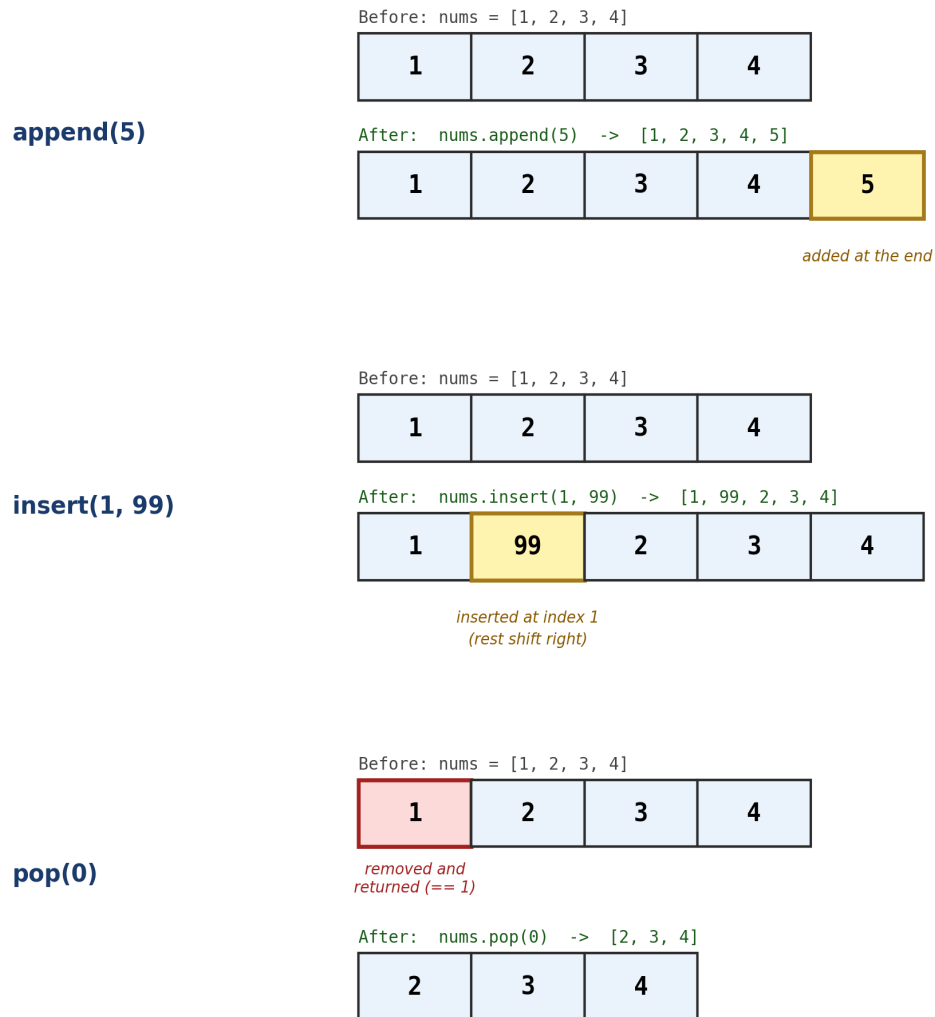


Figure 2: `append()` adds to the end, `insert()` adds at a specific index and shifts later elements right, and `pop()` removes an element by index and returns it.

4.1 `append(item)`

Adds a single item to the end of the list, in place.

```
nums = [1, 2, 3, 4]
nums.append(5)
print(nums)
```

Output

```
[1, 2, 3, 4, 5]
```

4.2 `insert(index, item)`

Inserts an item at a specific index, shifting later elements one position to the right.

```
nums = [1, 2, 3, 4]
nums.insert(1, 99)
print(nums)
```

Output

```
[1, 99, 2, 3, 4]
```

4.3 remove(value)

Removes the FIRST occurrence of a given VALUE. If the value does not exist, a `ValueError` is raised.

```
nums = [1, 2, 3, 2, 4]
nums.remove(2)
print(nums)
```

Output

```
[1, 3, 2, 4]
```

4.4 pop(index)

Removes the item at a given INDEX (the last item if no index is given) and RETURNS it, so you can capture it in a variable.

```
nums = [1, 2, 3, 4]
removed = nums.pop(0)
print(removed)
print(nums)

last_item = nums.pop()
print(last_item)
print(nums)
```

Output

```
1
[2, 3, 4]
4
[2, 3]
```

remove() vs. pop()

`remove(value)` searches for a VALUE and deletes its first match. `pop(index)` deletes whatever is sitting at a given POSITION and hands it back to you. Mixing these two up is a very common beginner mistake -- see Section 9.

4.5 sort()

Sorts the list in place (ascending by default) and returns None -- it does NOT return the sorted list, so do not write `nums = nums.sort()`.

```
nums = [4, 1, 3, 2]
nums.sort()
print(nums)

nums.sort(reverse=True)
print(nums)
```

Output

```
[1, 2, 3, 4]
[4, 3, 2, 1]
```

4.6 reverse()

Reverses the order of the list in place.

```
nums = [1, 2, 3, 4]
nums.reverse()
print(nums)
```

Output

```
[4, 3, 2, 1]
```

4.7 A few more useful methods

These are used less often than the six above, but are worth recognizing:

- `extend(other_list)` -- adds every element of another list to the end (unlike `append`, which would add the whole other list as a single nested item)
- `count(value)` -- returns how many times a value appears in the list
- `index(value)` -- returns the index of the first occurrence of a value (raises `ValueError` if missing, similar in spirit to `find()` returning -1 for strings, but NOT the same behavior -- see Section 9)
- `clear()` -- removes all elements, leaving an empty list `[]`
- `copy()` -- returns a shallow copy of the list (more on why this matters in Section 9)

Example

```
a = [1, 2, 3]
b = [4, 5]
a.extend(b)
print(a)

nums = [1, 2, 2, 3, 2]
print(nums.count(2))
print(nums.index(3))

nums.clear()
print(nums)
```

Output

```
[1, 2, 3, 4, 5]
3
3
[]
```

5. Membership: in and not in

The `in` and `not in` operators work on lists exactly as they did on strings in Chapter 4 -- they check whether a value exists in the sequence and return a boolean.

```
fruits = ["apple", "banana", "cherry"]

print("banana" in fruits)
print("mango" in fruits)
print("mango" not in fruits)
```

Output

```
True
False
True
```

One difference worth noting: `in` on a string checks for a SUBSTRING anywhere inside the text, while `in` on a list checks whether a value matches one of the list's elements EXACTLY.

6. Nested Lists

Because a list can hold any data type, it can hold OTHER lists. A "list of lists" is called a **nested list**, and is commonly used to represent grid-like or table-like data, such as a matrix.

Example

```
matrix = [[1, 2, 3],
          [4, 5, 6],
          [7, 8, 9]]

print(matrix)
print(matrix[1])      # the second inner list
print(matrix[1][2])   # the third item of the second inner list
```

Output

```
[[1, 2, 3], [4, 5, 6], [7, 8, 9]]
[4, 5, 6]
6
```

matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]

	col 0	col 1	col 2
row 0	1	2	3
row 1	4	5	6
row 2	7	8	9

matrix[1][2] == 6

outer index selects the ROW (a list); inner index selects the COLUMN (an item inside that row)

Figure 3: `matrix[1][2]` -- the first index picks a ROW (an inner list), and the second index picks a COLUMN (an item inside that row).

Because lists are mutable, you can also modify an element inside a nested list directly:

```
matrix[0][1] = 99
print(matrix)
```

Output

```
[[1, 99, 3], [4, 5, 6], [7, 8, 9]]
```

7. Iterating Over a List

A for loop lets you visit every element of a list, one at a time, without manually indexing.

Example

```
fruits = ["apple", "banana", "cherry"]

for fruit in fruits:
    print(fruit)
```

Output

```
apple
banana
cherry
```

Sometimes you need both the index AND the value while looping. The built-in `enumerate()` function gives you both at once. You will see it in full depth in Chapter 10 (Loops), but here is a quick preview:

```
fruits = ["apple", "banana", "cherry"]

for index, fruit in enumerate(fruits):
    print(index, fruit)
```

Output

```
0 apple
1 banana
2 cherry
```

8. List Comprehension (a First Look)

List comprehension is a compact way to build a new list from an existing iterable, all on one line. The basic pattern is `[expression for item in iterable]`. It is simply a shorthand for a for loop that builds up a list -- Chapter 10 (Loops) will cover this in much greater depth once you have spent more time with loops, but it is useful to be able to recognize it now.

The long way, with a for loop:

```
squares = []
for n in range(1, 6):
    squares.append(n * n)

print(squares)
```

Output

```
[1, 4, 9, 16, 25]
```

The same result with list comprehension:

```
squares = [n * n for n in range(1, 6)]  
print(squares)
```

Output

```
[1, 4, 9, 16, 25]
```

One more example, using a string:

```
word = "python"  
letters = [ch.upper() for ch in word]  
print(letters)
```

Output

```
['P', 'Y', 'T', 'H', 'O', 'N']
```

Don't worry if this feels unfamiliar

List comprehension combines a for loop and a list into one compact expression. If the syntax feels new right now, that is completely normal -- you have not formally covered for loops in depth yet. Revisit this section after Chapter 10 and it will click into place.

9. Common Beginner Mistakes

Mistake 1: IndexError on out-of-range access

Accessing an index that does not exist (for example `fruits[10]` on a 4-element list) raises `IndexError: list index out of range`. Just like with strings, slicing does NOT raise this error -- `fruits[0:100]` simply returns whatever elements exist.

Mistake 2: Confusing `remove()` with `pop()`

`remove(value)` deletes by VALUE; `pop(index)` deletes by POSITION and returns the removed item. Calling `nums.remove(0)` when you meant `nums.pop(0)` will delete the VALUE 0 (if present) instead of the element at index 0 -- and will raise a `ValueError` if 0 is not actually one of the values.

Mistake 3: Modifying a list while iterating over it

Adding or removing elements from a list WHILE a for loop is running over it can cause elements to be skipped or visited twice, because the loop tracks position by index as the list shifts underneath it. If you need to remove items based on a condition, build a new list (for example with a list comprehension) instead of modifying the original while looping over it.

```

nums = [1, 2, 3, 4, 5, 6]

# Risky: modifying nums while looping over it can skip elements
for n in nums:
    if n % 2 == 0:
        nums.remove(n)

# Safer: build a new list instead
nums = [1, 2, 3, 4, 5, 6]
odd_nums = [n for n in nums if n % 2 != 0]
print(odd_nums)

```

Output

```
[1, 3, 5]
```

Mistake 4: Shallow copy issues -- copy() vs. direct assignment

Writing `b = a` does NOT create a new list -- it makes `b` point to the exact same list object as `a`. Changing `b` will also change `a`. To get an independent copy, use `a.copy()` or `a[:]` instead.

```

a = [1, 2, 3]
b = a                # b is NOT a copy -- same list, two names
b.append(4)
print(a)             # a changed too!

c = a.copy()         # c IS an independent copy
c.append(99)
print(a)             # a is unaffected
print(c)

```

Output

```

[1, 2, 3, 4]
[1, 2, 3, 4]
[1, 2, 3, 4, 99]

```

Mistake 5: Forgetting that sort() modifies in place and returns None

`nums.sort()` sorts `nums` directly and gives back `None`. Writing `nums = nums.sort()` throws away your list and replaces it with `None`. If you want a NEW sorted list while keeping the original unchanged, use the built-in function `sorted(nums)` instead, which returns a new list.

Chapter 5 Recap

- Lists are written with square brackets and can mix data types; indexing, `len()`, and slicing all work the same way they did on strings in Chapter 4.
- Unlike strings, lists are **MUTABLE** -- elements can be changed, added, or removed after creation.

- `append()`, `insert()`, `remove()`, `pop()`, `sort()`, and `reverse()` are six essential list methods; `extend()`, `count()`, `index()`, `clear()`, and `copy()` round out the toolkit.
- `in` and `not in` test for membership on lists just as they did on strings.
- Nested lists (lists of lists) model grid-like data; `matrix[row][col]` accesses a single element.
- A `for` loop visits every element of a list in order; `enumerate()` gives you the index and the value together (full coverage in Chapter 10).
- List comprehension -- `[expression for item in iterable]` -- is a compact shorthand for building a list with a loop.

Looking ahead

Chapter 6 introduces tuples, an immutable alternative to lists -- useful whenever you want a collection of values that should never change after creation. Then Chapter 10 (Loops) will revisit iteration and list comprehension in much greater depth, building directly on the `for`-loop preview from this chapter.